



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

Game Programming

Fachhochschule Oberösterreich

Agenda

-
1. Introduction to Game Architecture
 2. Best Practices

- **Definition:** The high-level structure of a software system: how components are organised, how they communicate, and how they evolve over time

- **Why It Matters in Games**
 - Games are among the most complex real-time software systems:
 - 60–144 fps deadline every frame — no GC pauses, no stalls
 - Dozens of interacting subsystems (physics, AI, audio, rendering)
 - Large heterogeneous teams: artists, designers, programmers
 - Platform diversity: PC, consoles, mobile, VR

- **Architecture Is Not Design Patterns**
 - Patterns (Observer, Singleton, State...) are micro-level solutions
 - Architecture is the macro-level skeleton those patterns live inside

Game Architecture

Key Architectural Quality Attributes



Performance

Frames-per-second budget, memory bandwidth, CPU/GPU balance. Architecture defines your ceiling.



Scalability

Adding 100 more enemies should not require rewriting the rendering system. Systems scale independently.



Maintainability

Can a new team member add a feature without breaking five others? Clear interfaces and low coupling.



Modularity

Swap physics engines, audio middleware, or rendering backends without touching gameplay code.



Robustness

Handles invalid state gracefully. Crashes in one system shouldn't corrupt others. Especially critical in multiplayer.



Testability

Pure systems with no hidden state are unit-testable. This matters for tools, AI, and gameplay rules.

Game Architecture

Coupling & Cohesion: The Core Trade-off

Coupling

The degree to which one module depends on another.

Tight Coupling (Bad)

Physics directly calls `AudioManager.PlaySound()`
Rendering reads from AI state struct directly
Changing one breaks the other

Loose Coupling (Good)

Physics fires an event/message
Audio system subscribes and reacts
Systems are independently swappable

How to Achieve It

- Interfaces / abstract types
- Message bus / event system
- Dependency injection
- Service locator pattern

Cohesion

How focused and related the responsibilities of one module are.

Low Cohesion (Bad)

A 'GameManager' that handles input, saves, UI, audio, and AI.
Becomes a 2000-line God class.

High Cohesion (Good)

InputSystem reads input only.
SaveSystem serialises state only.
Each class has one reason to change.

SOLID Principle Applied

- S — Single Responsibility Principle
- O — Open/Closed Principle
- L — Liskov Substitution Principle
- I — Interface Segregation Principle
- D — Dependency Inversion

Game Architecture

Architectural Styles in Game Development

Layered

Engine → Framework → Game Layer → Scripting. Each layer only calls downward. Classic game engine organisation. Unity broadly follows this.

Component

Behaviour split into composable components attached to entities. Unity's MonoBehaviour model. Promotes reuse and composition over inheritance.

Event-Driven

Systems communicate through events/messages. Decouples producers from consumers. Used for UI, audio triggers, achievement systems.

Data-Oriented

Organise data for cache efficiency, not for objects. ECS (Entity-Component-System) is the dominant form. Unity DOTS, EnTT (Unreal), flecs.

Client-Server

Authoritative server holds true game state; clients predict and reconcile. The foundation of every online multiplayer game.

Microkernel

Minimal core (plugin manager + message bus) with feature plugins. Unreal Engine follows this for its module system.

Game Architecture

Monolithic vs Modular Architecture

Aspect	Monolithic	Modular
Compile time	One large build — slow iteration	Incremental — only changed modules
Coupling	High — direct references everywhere	Low — defined interfaces
Testability	Hard — need full runtime	Easy — test modules in isolation
Team scaling	Merge conflicts constant	Teams own separate modules
Runtime cost	Single binary — can be faster	Slight overhead from interface calls
Game examples	Early Unity projects, game jams	Unreal, mature Unity projects, FMOD
Best for	Prototypes, small teams, game jams	AAA, long-lived projects, large teams

Game Architecture

The Game Loop: Heart of Every Game

→ The Core Pattern

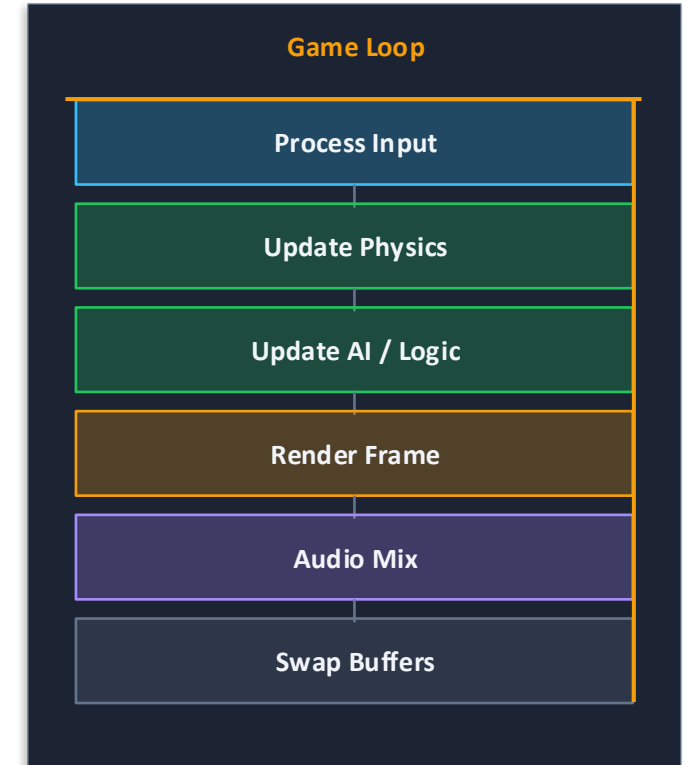
- Every game runs a loop: sample input → update world → render → repeat
- The architectural decisions around this loop define performance and feel

→ Variants

- Fixed Timestep: update runs at fixed rate (e.g. 50 Hz physics)
- Variable Timestep: update runs as fast as possible, delta-time scaled
- Semi-Fixed: accumulator pattern — most common modern approach

→ Unity's Loop

- Update(): per-frame gameplay logic
- FixedUpdate(): physics, at fixed 50 Hz default
- LateUpdate(): camera, finalise transforms
- OnPreRender / OnPostRender: graphics hooks



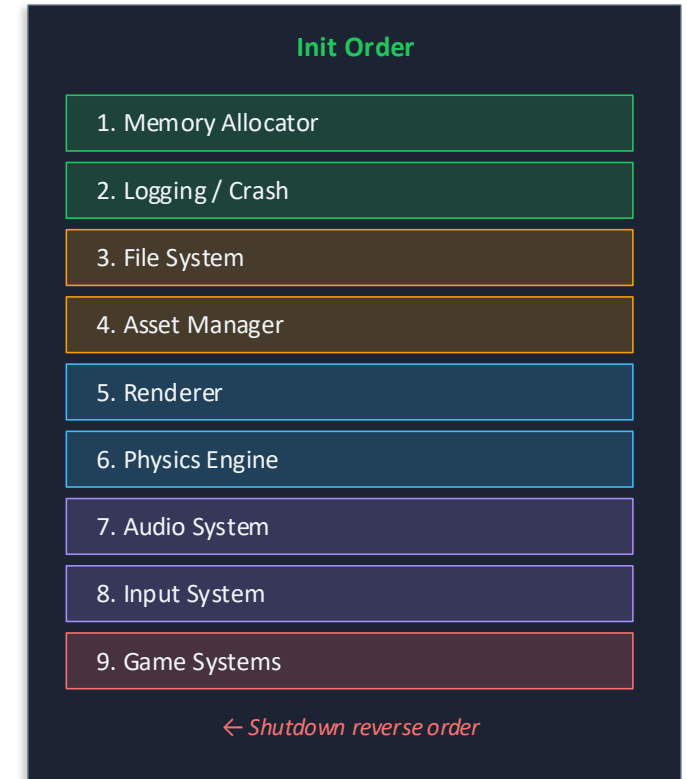
Game Architecture

Subsystem Initialisation & Dependency Order

- **Challenge:** Subsystems depend on each other
- Audio needs memory allocator
 - Renderer needs window
 - Physics needs memory and renderer for debug draw
 - Order matters

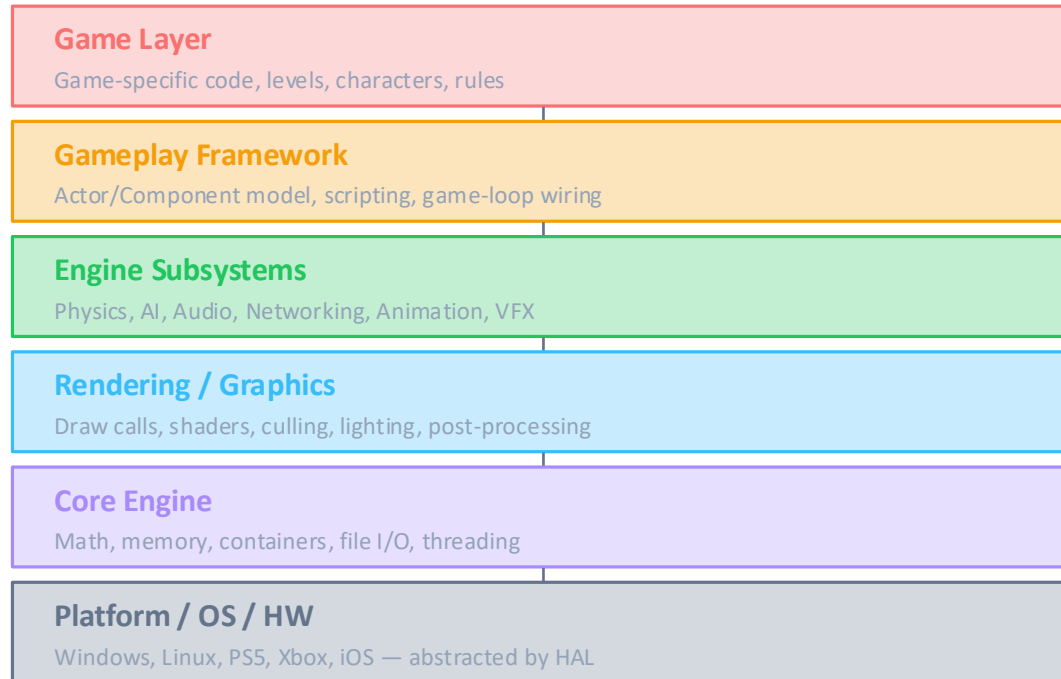
→ **Initialization Strategies**

- Explicit ordered init: hand-written order in main.cpp / Bootstrap
- Dependency injection: constructor declares what it needs
- Lazy init: subsystem initialises on first use (careful with order!)
- Unity Scripting: Script Execution Order (Project Settings)



Game Architecture

Game Engine Layer Model



Direction of calls/dependencies (game calls engine, never reverse)

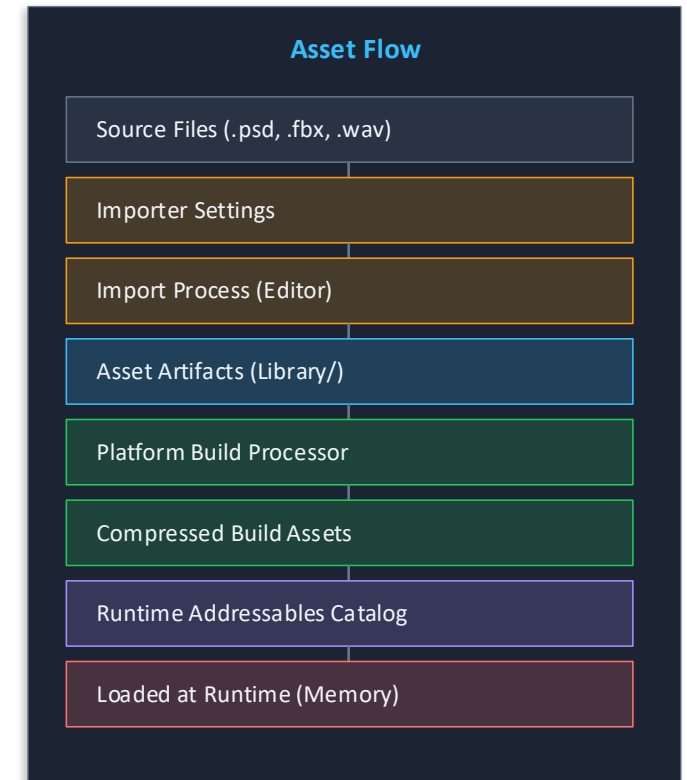
Unity Mapping	
Game	Your C# MonoBehaviours
Framework	MonoBehaviour, GameObject, SceneManager
Subsystems	Rigidbody, AudioSource, NavMesh, Animator
Rendering	URP/HDRP, Shader Graph, Camera, Batcher
Core	Unity C++ runtime, JobSystem, Burst, NativeArray
Platform	Player Build, Platform Backends, IL2CPP

→ What is the Asset Pipeline?

- The process of taking source art, audio, and data files and converting them into game-ready runtime formats
- Architectural decisions here massively affect build times and team workflow

→ Unity's Pipeline

- AssetImporter: each asset type has an importer (TextureImporter, AudioImporter...)
- AssetDatabase: tracks all assets, GUIDs, dependencies, platform variants
- Addressables: async runtime loading by address, not direct reference



Game Architecture

Scene Architecture & the GameObject Hierarchy

→ Unity Scene Graph

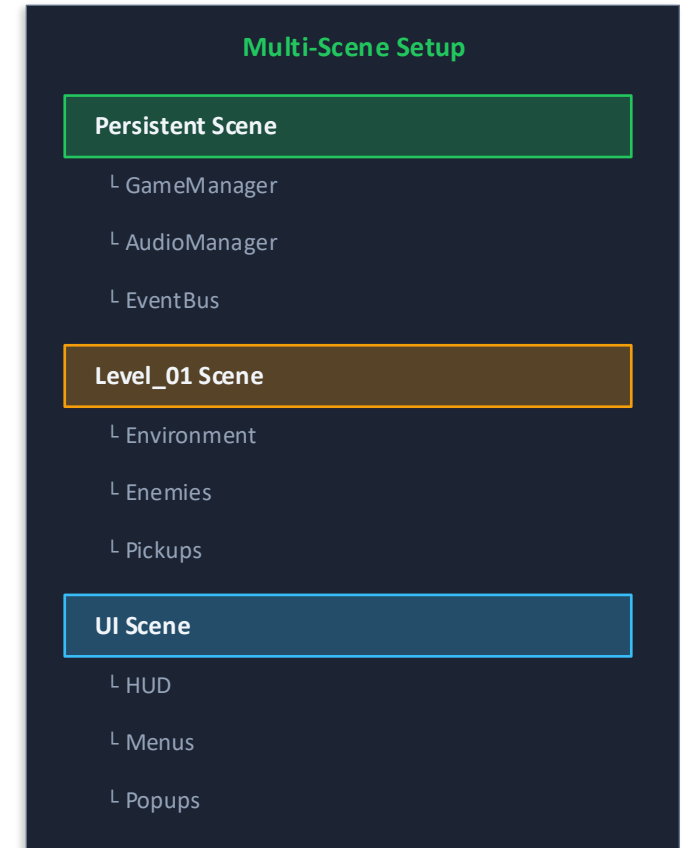
- A tree of GameObjects, each with a Transform
- Parent transforms propagate to children
- Flat hierarchies render faster than deep ones

→ Hierarchy Design Rules

- Limit depth to 4–5 levels max -> deeper = slower matrix multiplication
- Don't parent dynamic objects under moving objects unless needed
- Group by function: [Environment], [Characters], [UI], [Systems]
- Keep systems (GameManager, EventBus) out of the scene hierarchy

→ Multi-Scene Architecture

- LoadSceneMode.Additive: load multiple scenes simultaneously
- Persistent scene (DoNotDestroyOnLoad): managers, player
- Level scenes: streamed in/out as player traverses world
- UI scene: overlay loaded additively, always on top



Game Architecture

Memory in Games



Cache Hierarchy

L1 cache ~4 ns, RAM ~100 ns, SSD ~100 μ s. Cache-friendly access patterns (sequential, small structs) are 25 $\times$ faster than random pointer chasing.



Heap Fragmentation

Frequent alloc/free creates holes. A game that runs fine for 10 min may crash after 2 hours. Pools and arenas prevent this.



GC Spikes (Unity)

C# garbage collector pauses the thread when collecting. At 60 fps you have 16.6 ms. A 5 ms GC spike drops you to 45 fps.



Fixed Budgets

Console platforms have strict memory limits (PS5: 13.5 GB, Switch: 4 GB). Budget is allocated per system at architecture phase.

Game Architecture

GC Avoidance Strategies in Unity

Anti-Pattern (Allocates GC)	Better Alternative
<code>new Vector3[n]</code> every frame	Reuse cached array; <code>NativeArray</code> with <code>Allocator.Temp</code>
LINQ in <code>Update()</code> (<code>.Where</code> , <code>.Select</code>)	Manual loop; <code>Unity.Collections</code> , <code>Span<T></code>
string + concatenation in loop	<code>StringBuilder</code> , <code>string.Format</code> , or interpolation once
foreach on Dictionary	Use <code>Dictionary.TryGetValue</code> , avoid enumerator alloc
<code>GetComponent<T>()</code> in <code>Update()</code>	Cache reference in <code>Awake()</code> / <code>Start()</code>
Instantiate/Destroy every frame	Object pooling (<code>UnityEngine.Pool</code>)
<code>AddComponent</code> at runtime	Disable/enable pre-placed components
<code>UnityEvent</code> with many listeners	C# events or direct method calls

Game Architecture

OOP vs ECS: A Paradigm Shift

→ Object-Oriented (Traditional)

- Entities are objects with data AND behaviour
- A Player class has position, health, inventory, input handling, rendering all in one
- Pro: Natural mental model, easy to start
- Con: Deep inheritance hierarchies, virtual call overhead
- Con: Data scattered across heap -> terrible cache behaviour
- Con: Hard to parallelise (shared mutable state)

→ Entity Component System (ECS)

- Entities are just integer Ids
- Components are pure data structs
- Systems contain behaviour and operate on matching component queries
- Pro: Data laid out contiguously -> cache-friendly
- Pro: Systems trivially parallelisable (data ownership explicit)
- Pro: Composition over inheritance -> mix any components
- Con: Steeper learning curve, different mental model

Memory Layout

OOP (AoS — Array of Structs):

```
[Pos|Vel|HP|Mesh|Audio]  
[Pos|Vel|HP|Mesh|Audio]  
[Pos|Vel|HP|Mesh|Audio]
```

Cache miss when only position needed!

ECS (SoA — Struct of Arrays):

```
Positions: [P P P P P P]  
Velocities:[V V V V V V]  
HP:        [H H H H H H]
```

Perfect cache utilisation for movement system!

Game Architecture

Open World Streaming Architecture

→ Problem:

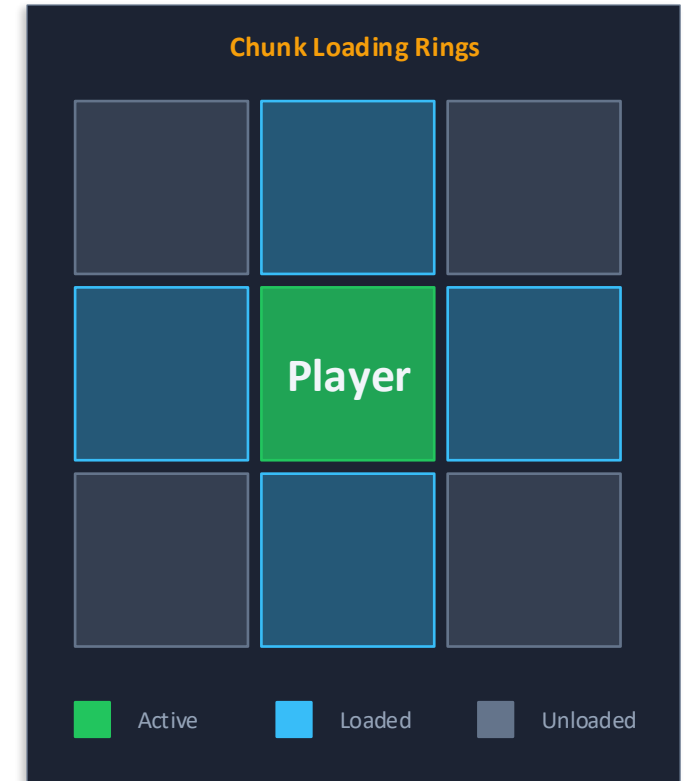
- A 10 km² open world cannot fit in VRAM or RAM simultaneously
- The engine must stream assets in and out as the player moves

→ Strategies

- Cell/Chunk streaming: divide world into cells; load ring around player
- Portals: load adjacent rooms when crossing a portal trigger
- Level-of-Detail (LOD): lower-detail meshes for distant objects
- Imposters: billboard sprites replace distant 3D objects (trees, rocks)

→ Unity Tools

- Addressables + async LoadSceneAsync for cell-based streaming
- Scene Management API: LoadSceneMode.Additive / UnloadSceneAsync
- LOD Group component: automatic LOD switching by screen coverage
- Unity Streaming Assets: raw bytes streamed from disk



→ Prefabs as Architecture

- Prefabs are more than convenience -> they define your entity hierarchy, component composition, and default values
- Poor prefab design cascades into maintainability problems

→ Nested Prefabs

- Prefab Variants: inherit from a base prefab, override specific values
- Nested Prefabs: a prefab contains instances of other prefabs
- Changes to base propagate to all variants automatically

→ Design Principles

- Single-responsibility prefab: one prefab per logical entity type
- No scene-specific hardwiring: reference by interface, not scene name
- Data separation: stats in ScriptableObjects, not prefab fields
- Avoid deep nesting: prefer flat prefabs + runtime parent assignment

Prefab Variant Tree

Enemy (Base Prefab)

└─ Zombie (Variant)

 └─ FastZombie

 └─ TankZombie

└─ Robot (Variant)

 └─ FlyingRobot

 └─ ShieldRobot

└─ Boss (Variant)

Game Architecture

Multiplayer Architecture Models

Dedicated Server (Authoritative)

Example: CS2, Valorant, Overwatch

- ✓ Server is ground truth -> no cheating
- ✓ Works for any player count
- ✓ Clients are thin: input → server → state back
- ✗ Server costs money to run
- ✗ Higher latency

Peer-to-Peer (Lockstep)

Example: Age of Empires, early Warcraft

- ✓ No server costs
- ✓ Deterministic -> same inputs = same state
- ✓ Classic RTS model (Age of Empires, StarCraft)
- ✗ All peers must keep up with slowest
- ✗ Desync = game over
- ✗ Cheating possible

Client-Hosted (Listen Server)

Example: Minecraft Java, early Unity games

- ✓ Easy to implement
- ✓ Host has zero latency advantage
- ✓ Works for LAN and small groups
- ✗ Host has unfair advantage
- ✗ Game ends if host disconnects

Network Data Architecture: What to Send

→ Core Principle: Bandwidth Is Scarce

- Typical game bandwidth budget: 50–200 KB/s per player
- Every byte counts
- Architecture must define what gets replicated, at what frequency, and with what priority

→ Replication Strategies

- Full State: send entire world every tick — simple, expensive, for LAN only
- Delta Compression: send only changed fields since last ack — standard AAA
- Interest Management: only send data relevant to the client's view area
- Quantisation: compress float positions to 16-bit int — 50% savings

Game Architecture

Frame Budget: Architecture for Performance

60 fps

16.67 ms/frame

30 fps

33.33 ms/frame

120 fps

8.33 ms/frame

90 fps VR

11.11 ms/frame

Typical 60fps Budget Allocation (16.67 ms)

5.5ms
Rendering / GPU

2.0ms
Physics

2.5ms
AI / Logic

1.0ms
Audio

1.5ms
GC / Misc

4.2ms
Headroom

Game Architecture

Draw Calls, Batching and GPU Architecture

→ Draw Call Problem

- Each draw call = CPU sets GPU state + issues command
- Setup overhead is expensive
- 1000 draw calls on mobile = framerate disaster

→ Batching Strategies

- Static Batching: combine static meshes into one large VB at build time
 - No CPU cost at runtime. Large memory cost.
- Dynamic Batching: runtime combine small meshes with same material
 - Limited to <900 vertices
 - Unity does this automatically
- GPU Instancing: render 1000s of identical meshes in one draw call
- SRP Batcher: Unity URP/HDRP
 - Batches CBUFFERS -> not meshes. Huge speedup for many materials

Batch Count Targets

PC

< 2000 draw calls

Console

< 1500 draw calls

VR

< 200 draw calls

Mobile

< 100 draw calls

Game Architecture

Profiling Tools & Workflow in Unity



Unity Profiler

Window → Analysis → Profiler. CPU, GPU, Memory, Audio, Rendering tracks. Deep profile mode shows per-method cost. Always profile in Development Build.



Memory Profiler

Package: com.unity.memoryprofiler. Shows heap, native objects, GC allocations, reference graph. Essential for leak detection.



Frame Debugger

Window → Analysis → Frame Debugger. Steps through every draw call in a frame. Shows why batching broke, overdraw, culling.



Burst Inspector

Jobs → Burst → Open Inspector. Shows generated SIMD assembly for each job. Verify auto-vectorisation and alias assumptions.



RenderDoc / PIX

External GPU profiler (Windows/Linux). Pixel-level GPU timing, shader execution, texture bandwidth. For rendering engineers.



Custom Markers

```
using Unity.Profiling;  
ProfilerMarker _m = new("MySystem");  
_m.Begin(); ... _m.End();  
Shows in Profiler timeline.
```

Game Architecture

Build Pipeline Architecture

→ CI/CD for Games

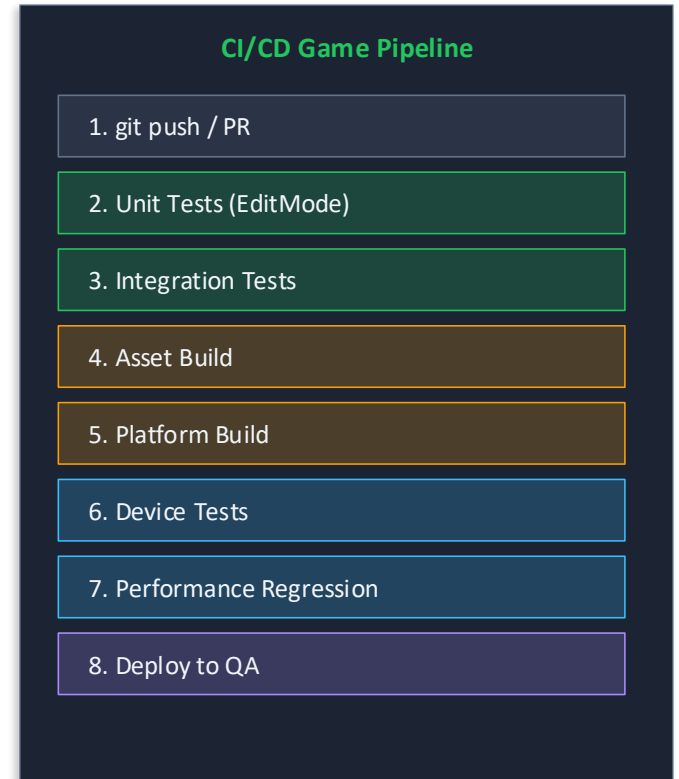
- Automated build pipelines ensure every commit is validated, packaged, and tested
- The same as in enterprise software, but with longer build times 😊

→ Unity Build Pipeline

- Addressables Build: asset bundles, catalog, content update packages
- BuildPlayerOptions: scripted builds via Editor scripting (BuildPipeline.BuildPlayer)
- Platform specifics: Android AAB, iOS Xcode archive, PS5 .pkg generation
- IL2CPP vs Mono: IL2CPP required for iOS, best on all platforms for performance

→ Automation Tools

- Unity Cloud Build / Unity DevOps: hosted CI/CD for Unity projects
- Jenkins / GitHub Actions: general-purpose CI runners with Unity batchmode
- GameCI (game.ci): free GitHub Actions workflows for Unity builds
- Fastlane + Unity: automate iOS/Android signing and store upload



Game Architecture

Game State Management Architecture

→ Problem with Global State

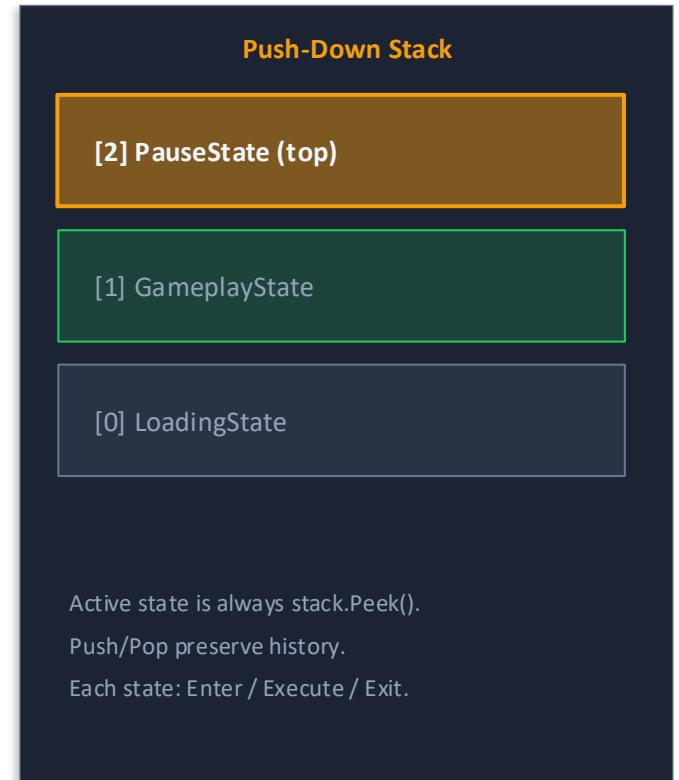
- Games have many states: MainMenu, Loading, Gameplay, Paused, GameOver, Credits
- Ad-hoc if/else chains in a GameManager god class become unmaintainable

→ Hierarchical State Machine (HSM)

- States can have sub-states (Gameplay → Combat, Exploration, Cutscene)
- Parent states handle common transitions; child states specialize
- Animancer, NodeCanvas, Behaviour Designer: Unity HSM tools

→ Push-Down Automata (Stack)

- Push state on stack: Gameplay → push Pause (gameplay underneath)
- Pop returns to previous: Pause → pop → resume Gameplay
- Ideal for pause menus, dialogue, inventory overlays



Game Architecture

Common Game Architecture Anti-Patterns



God Class / GameManager

One class that does everything. Input, AI, UI, audio, saves. The #1 cause of unmaintainable game codebases. Break it up early.



Singletons Everywhere

Each system is a Singleton. Creates hidden dependencies and makes testing impossible. Prefer dependency injection or Service Locator.



Scene Coupling

Code that has hard-coded scene names, `GameObject.Find()`, or direct Inspector references across scenes. Use Addressables and events.



Inheritance Deep Hierarchy

Player > Character > LivingThing > Entity > MonoBehaviour. 10-level inheritance breaks with any requirement change. Prefer composition.



Update() Overuse

1000 MonoBehaviours each with an `Update()` call. Unity iterates all — huge overhead. Use custom manager that explicitly calls registered objects.



Premature Optimisation

Spending 3 days on micro-optimisations before profiling. The bottleneck is almost never where you think. Profile first, always.

Game Architecture

Architecture Decision Records (ADRs)

→ What Is an ADR?

- A short document capturing an architectural decision: context, options considered, decision made, and consequences
- Living documentation of why the architecture is the way it is

→ Format

- Title: short imperative phrase (Use ECS for bullet entities)
- Status: Proposed / Accepted / Superseded
- Context: the situation that forces a decision
- Options Considered: 2–3 alternatives with pros/cons
- Decision: chosen option and why
- Consequences: positive, negative, neutral outcomes

→ Why ADRs Matter in Games

- Team turnover is high — ADRs preserve institutional knowledge
- Post-mortems often reveal forgotten reasoning behind bad decisions
- Forces teams to think through alternatives before committing

ADR Example

ADR-003: Use FMOD

Status: Accepted

Context:

Need adaptive music;
Unity Mixer insufficient

Options:

- A) Unity Mixer (free)
- B) FMOD (licence)
- C) Wwise (complex)

Decision: B (FMOD)

Designer can iterate
without programmer help.

Consequence:

- +Designer autonomy
- Additional £ licence
- Integration work

Game Architecture

Summary & Key Takeaways



Architecture is about trade-offs

Coupling, cohesion, scalability, and performance all pull in different directions. Document your decisions (ADRs).



Master the game loop

Fixed timestep, accumulator pattern, subsystem init order — these are foundations everything else builds on.



Know your engine internals

Managed vs native, the Job system, Burst, render pipelines — knowing the internals makes you a better architect.



Memory is architecture

GC, pools, allocators, cache layout — memory decisions made at architecture time define your runtime ceiling.



ECS is the future for large scale

DOTS/ECS is not a replacement for MonoBehaviour — it's the right tool when you hit 10k+ entities.



Network architecture defines everything

Server authority, prediction, rollback — these must be designed in from day one, not bolted on later.

Thanks for your Attention



UNIVERSITY
OF APPLIED SCIENCES
UPPER AUSTRIA

FH Oberösterreich

University of Applied Sciences Upper Austria

Digital Media Department

Media Technology & Design

Digital Arts

Interactive Media



Dr. Andreas Riegler

Professor of Interactive Systems

Digital Media Department

E Andreas.Riegler@fh-hagenberg.at